

eda_gym_exercice_dataset_v1.0

November 6, 2025

1 Gym_Exercise_Dataset.csv

1.1 Chargement et inspection initiale

1.1.1 Objectif

Vérifier que le fichier `Gym_Exercise_Dataset.csv` est correctement chargé et obtenir une vue d'ensemble : - dimensions du jeu de données (nombre de lignes et de colonnes), - typage des variables (numériques, catégorielles, textuelles), - détection des valeurs manquantes et doublons, - cohérence générale du corpus avant l'analyse statistique.

Cette étape permet de valider la structure et la qualité du jeu de données avant toute exploration descriptive ou visuelle.

1.1.2 Interprétation attendue

- Le dataset doit contenir une liste d'exercices accompagnés de leurs descriptions (`Title`, `Desc`) et de métadonnées associées (`Type`, `BodyPart`, `Equipment`, `Level`, `Rating`, `RatingDesc`).
- Les types doivent être correctement identifiés : **textuels** pour les descriptions et catégories, **numériques** pour les éventuelles notes.
- L'absence de valeurs manquantes et de doublons confirmera la fiabilité du corpus.
- Les noms de colonnes doivent être homogènes et cohérents pour permettre les traitements automatiques à venir.

1.1.3 Importation des librairies essentielles

```
[11]: import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import kagglehub

from scipy import stats
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# === Localiser automatiquement le package 'themes' en remontant l'arborescence
↳===
from pathlib import Path
```

```

import sys

root = Path.cwd()          # ex: .../DuckDB/eda/life_style_data
themes_root = None

for p in [root, *root.parents]:  # remonte: life_style_data -> eda -> DuckDB
    ↪-> ...
    if (p / "themes").is_dir():
        themes_root = p
        break

if themes_root is None:
    raise FileNotFoundError(
        f"Impossible de trouver le dossier 'themes' en partant de {root}. "
        "Vérifie l'arborescence (il doit exister un dossier 'themes/' quelque
    ↪part au-dessus)."
    )

if str(themes_root) not in sys.path:
    sys.path.insert(0, str(themes_root))

print(" Ajouté au PYTHONPATH :", themes_root)
print(" Contenu de", themes_root, ":", [x.name for x in (themes_root).
    ↪iterdir()])

from themes.train_me import set_trainme_theme

```

Ajouté au PYTHONPATH : c:\Users\fbac\Desktop\Projets\Dev\Projet Alyra\DuckDB
 Contenu de c:\Users\fbac\Desktop\Projets\Dev\Projet Alyra\DuckDB : ['.conda',
 '.vscode', '.~lock.Exploration des données - v1.1.odt#', 'datasets',
 'duckdb.exe', 'eda', 'Exploration des données - v1.0.odt', 'Exploration des
 données - v1.0.pdf', 'Exploration des données - v1.1.odt', 'themes']

1.1.4 Configuration d’affichage pour plus de lisibilité

```

[12]: pd.set_option('display.max_columns', None)
      pd.set_option('display.width', 120)

```

1.1.5 Thème “TrAIIn.me”

Il applique un fond dark propre, une grille discrète, des ticks lisibles, et une palette bleus/cians cohérente.

```

[13]: # =====
      # Thème TrAIIn.me (Seaborn/Matplotlib)
      # =====

```

```
# Booléen de contrôle : True = thème TrAI.n.me (dark), False = style clair
↪ "ticks"
USE_DARK_THEME = True # modifie ici selon le contexte

# Activer le thème au début du notebook :
palette_trainme = set_trainme_theme(context="talk", font_scale=1.05,
↪ use_dark=True)
palette_trainme
```

[13]: ['00E5FF', '#00BFFF', '#0A84FF', '#1E90FF', '#00FFFF', '#64D8FF']

1.1.6 Chargement du dataset

```
[14]: # Téléchargement du dataset depuis Kaggle
dataset_path = kagglehub.dataset_download("niharika41298/gym-exercise-data")

# Recherche du fichier CSV dans le dossier téléchargé
csv_files = [f for f in os.listdir(dataset_path) if f.endswith(".csv")]
print("Fichiers trouvés :", csv_files)

# Construction du chemin complet vers le CSV
file_path = os.path.join(dataset_path, csv_files[0])

# Lecture du fichier CSV
df = pd.read_csv(file_path)

# Informations de confirmation
print("Dataset chargé avec succès !")
print(f"Dimensions : {df.shape[0]} lignes et {df.shape[1]} colonnes\n")
```

Warning: Looks like you're using an outdated `kagglehub` version, please consider updating (latest version: 0.3.13)

Fichiers trouvés : ['megaGymDataset.csv']

Dataset chargé avec succès !

Dimensions : 2918 lignes et 9 colonnes

Fichiers trouvés : ['megaGymDataset.csv']

Dataset chargé avec succès !

Dimensions : 2918 lignes et 9 colonnes

1.1.7 Aperçu général des premières lignes

```
[15]: display(df.head())
```

Unnamed: 0	Title
↪ Desc	Type BodyPart \

```

0          0          Partner plank band row  The partner plank band row is an
↳abdominal exe... Strength Abdominals
1          1  Banded crunch isometric hold  The banded crunch isometric hold is
↳an exercis... Strength Abdominals
2          2          FYR Banded Plank Jack  The banded plank jack is a
↳variation on the pl... Strength Abdominals
3          3          Banded crunch  The banded crunch is an exercise
↳targeting the... Strength Abdominals
4          4          Crunch  The crunch is a popular core
↳exercise targetin... Strength Abdominals

```

	Equipment	Level	Rating	RatingDesc
0	Bands	Intermediate	0.0	NaN
1	Bands	Intermediate	NaN	NaN
2	Bands	Intermediate	NaN	NaN
3	Bands	Intermediate	NaN	NaN
4	Bands	Intermediate	NaN	NaN

1.1.8 Informations générales sur le typage et la complétude

```
[16]: df_info = df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2918 entries, 0 to 2917
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Unnamed: 0      2918 non-null  int64
1   Title           2918 non-null  object
2   Desc            1368 non-null  object
3   Type            2918 non-null  object
4   BodyPart        2918 non-null  object
5   Equipment       2886 non-null  object
6   Level           2918 non-null  object
7   Rating          1031 non-null  float64
8   RatingDesc      862 non-null   object
dtypes: float64(1), int64(1), object(7)
memory usage: 205.3+ KB

```

1.1.9 Statistiques descriptives des variables numériques

```
[17]: display(df.describe().T)
```

	count	mean	std	min	25%	50%	75%	max
Unnamed: 0	2918.0	1458.50000	842.498368	0.0	729.25	1458.5	2187.75	2917.0
Rating	1031.0	5.91969	3.584607	0.0	3.00	7.9	8.70	9.6

1.1.10 Analyse des valeurs manquantes et doublons

```
[18]: missing_values = df.isnull().sum().sort_values(ascending=False)
      duplicates = df.duplicated().sum()

      print("\n--- Valeurs manquantes par colonne (top 10) ---")
      display(missing_values.head(10))

      print(f"\nNombre total de doublons détectés : {duplicates}")
```

--- Valeurs manquantes par colonne (top 10) ---

RatingDesc	2056
Rating	1887
Desc	1550
Equipment	32
Unnamed: 0	0
Title	0
Type	0
BodyPart	0
Level	0

dtype: int64

Nombre total de doublons détectés : 0

1.1.11 Liste complète des colonnes disponibles

```
[19]: print("\n--- Liste des colonnes du dataset ---")
      print(df.columns.tolist())
```

--- Liste des colonnes du dataset ---

```
['Unnamed: 0', 'Title', 'Desc', 'Type', 'BodyPart', 'Equipment', 'Level',
 'Rating', 'RatingDesc']
```